

System Overview

This project aims to build a comprehensive, enterprise-grade **algorithmic trading system** from the ground up. The platform's core philosophy is a "Best-of-Breed" integration strategy, where the best open-source project for each major component is selected and merged to create a powerful, modular foundation.

The architecture is designed as a set of distinct microservices that communicate through a centralised API bridge. The system's standout feature is a sophisticated **Agentic AI Assistant**, which acts as the "brain" of the platform, allowing users to manage trading, research, and analysis using natural language commands.

Core Technology Stack

The system is built on a modern, open-source technology stack:

- **Core Trading Engine: QuantConnect LEAN** is the institutional-grade engine for backtesting, live trading, data management, and multi-asset support.
- **Agentic AI Assistant:** A combination of **LangChain** (as the core framework for Agentic RAG and agent creation) and **TradingAgents** (providing a finance-specific foundation with specialised agent roles).
- **User Interface:** A modern web application built with **Next.js** will serve as the unified "cockpit" for all user interactions.
- **Options Analytics Engine: QuantLib** is used for all sophisticated options pricing, risk analytics, and complex derivative calculations.
- **API & Orchestration Layer:** A custom **FastAPI** application serves as the central bridge between the frontend, the AI assistant, and the backend trading engine. For more complex workflows, **LangGraph** may be used.
- **Database:** The system uses **PostgreSQL** with the **pgvector** extension enabled, allowing it to function as an all-in-one database for both traditional relational data and vector embeddings for the AI. **Qdrant** is also used for high-performance vector search.
- **Observability Stack:** **Prometheus** and **Grafana** are integrated for comprehensive production monitoring and dashboarding.

Key Features and Capabilities

- **Trading and Order Management:** The system supports multiple asset classes (stocks, options, futures, forex, crypto) and connects to various brokers, starting with Interactive Brokers. It will handle basic order types (Market, Limit) as well as advanced and algorithmic orders (VWAP, TWAP).
- **AI-Powered Strategy Development:** Users can develop strategies through multiple interfaces:
 - A **No-Code Strategy Builder** with a visual drag-and-drop interface.
 - The **Agentic AI Assistant**, which can generate strategy code from natural language prompts.
 - A traditional Python-based studio environment.
- **Agentic AI Assistant:** This is the core user-interaction model. The assistant uses an Agentic **Retrieval-Augmented Generation (RAG)** pipeline to process and query user-uploaded documents. It is composed of a collaborative network of specialised agents (Analyst, Researcher, Risk Manager, Compliance, etc.) that can perform tasks by calling the system's API. Users can ask the AI to run a backtest, get portfolio status, or even place live trades.
- **Enterprise Security and Risk Management:** The platform is built with a **Zero-Trust architecture** and includes a real-time risk hub for pre-trade checks and account-level circuit breakers. It also features role-based access control (RBAC) and immutable audit trails for compliance.

5-Phase Development Plan

The project is structured into a clear 5-phase roadmap:

1. **Phase 1: Foundational Setup:** Establishes the core infrastructure using Docker, sets up the QuantConnect LEAN engine, and deploys the PostgreSQL database with the pgvector extension.
2. **Phase 2: API Bridge:** Builds the crucial FastAPI wrapper around the LEAN engine, enabling programmatic backtesting via a /backtest endpoint.
3. **Phase 3: AI Assistant MVP:** Brings the initial AI assistant to life by integrating LangChain and TradingAgents, building the Agentic RAG pipeline, and creating the first AI "tools" to call the backtest API. It also uses the Qdrant Python client to store the resulting text embeddings in a vector database.
4. **Phase 4: Full User Experience:** Develops the custom Next.js frontend, integrates TradingView charts, and connects the system to a paper trading and a live trading account, expanding the API and AI tools to manage a live portfolio.
5. **Phase 5: Advanced Features & Enterprise Readiness:** Layers in the most sophisticated features, including the QuantLib integration for options, the No-Code Strategy Builder, the full observability stack, and the comprehensive enterprise security and risk guardrails.

The "Best-of-Breed" Integration Strategy

The most effective strategy is to select the best project for each major component of your system and merge them to create a powerful, modular foundation. Each component will operate as a distinct service, communicating through a centralised API.

Core Technology Stack:

- **Core Trading Engine: QuantConnect LEAN**
 - **Role:** The undisputed choice for the institutional-grade engine handling backtesting, live trading, data management, and multi-asset support.
 - **Rationale:** It offers the most comprehensive feature set, extensive broker and data integrations, Python compatibility, and a large, active community, covering the bulk of your core infrastructure needs.
- **Agentic AI Assistant: TradingAgents, LangChain & LangGraph**
 - **Role:** This combination will form the "brain" of the system.
 - **Rationale:** TradingAgents provides a finance-specific foundation with specialised agent roles (analyst, researcher, risk manager) that directly map to your requirements. LangChain will serve as the underlying framework for building the Agentic Retrieval-Augmented Generation (Agentic RAG) capabilities, document processing, and the tool integration that powers the agents, as recommended in your features document. LangGraph will manage the complex, asynchronous workflows between the AI Agents, Multi Agents, Agentic AI, data, and trading services and act as an Orchestrator.
- **Options Analytics Engine: QuantLib**
 - **Role:** The go-to open-source library for all sophisticated options pricing, risk analytics, and calculations for complex derivatives.
 - **Rationale:** It is the premier library for quantitative finance, providing tools that are a powerful open-source alternative to proprietary systems. It will be integrated as a specialised service.
- **User Interface: Next.js**
 - **Role:** The unified "cockpit" that interacts with all backend services via a centralised API.
 - **Rationale:** This is the modern standard for high-performance web applications and is specified in your features document.

Summary of Feature Coverage

This integration strategy provides a strong foundation, but significant custom development is required to meet all enterprise-grade requirements.

- **Covered by Foundation Repositories:**
 - **QuantConnect LEAN:** Provides the core backtesting engine, multi-broker connectivity (Interactive Brokers, Alpaca, etc.), support for numerous asset classes (stocks, options, futures, crypto), basic order types, historical data access, and a framework for live trading.
 - **TradingAgents:** Offers a multi-agent framework tailored for finance with predefined roles like analyst, researcher, and risk manager.
 - **LangChain:** Provides the essential toolkit for building Agentic RAG pipelines, creating AI agents, and integrating with language models and various data sources.
 - **LangGraph:** Manages complex, asynchronous workflows between the AI Agents, Multi Agents, Agentic AI, data, and trading services and acts as an Orchestrator.
 - **QuantLib:** Delivers advanced financial modeling for options pricing (Greeks, exotics), volatility surfaces, and interest rate curves.
- **Requires Custom Development/Integration:**
 - **API & Orchestration Layer:** A custom FastAPI wrapper will serve as the central bridge between services. For more complex workflows, an orchestration engine like LangGraph is recommended.
 - **No-Code Strategy Builder:** The front-end drag-and-drop interface and the backend logic to interpret its output will be custom-built.
 - **Advanced Features:** Most advanced features require custom logic, including the AI-powered opportunity detection, post-trade analytics, enterprise security guardrails, advanced risk management hub, and the complete user interface.
 - **Database & Data Pipelines:** While libraries like Unstructured.io and databases like Qdrant and PostgreSQL will be used, the pipelines for data ingestion, resiliency, and fusion are custom logic you will build.

Revised 5-Phase Development Plan: Merging Open-Source Repositories

Here is the revised, 5-phase plan that integrates these repositories to build your complete system.

Prompt for Phase 1:

You are building an enterprise algorithmic trading system. Your task is to set up the foundation for Phase 1.

1. Create a master Git repository structure with separate directories: /lean_engine, /ai_assistant, and /frontend.
2. Generate a docker-compose.yml file. This file must define two services:
 - o A lean_engine service using the official QuantConnect LEAN Docker image.
 - o A database service using the postgres:latest image. Crucially, this service must have the **pgvector** extension enabled to handle both relational data and vector embeddings.
3. Write a Python script within the /lean_engine directory that uses the LEAN engine to run a simple backtest (e.g., a basic moving average crossover strategy on AAPL) using data from Yahoo Finance.
4. The script should demonstrate that the LEAN engine can successfully fetch data, run the backtest, and output the results to the console. The end goal is a running Docker environment with LEAN and a vector-ready PostgreSQL database.
5. Also refer to the following details and decide & plan accordingly.

Phase 1: Foundational Setup & Core Engine Integration

Objective: To establish the project's core infrastructure and get the main trading engine up and running with its essential data connections.

Integration Plan & Actions:

1. **Set Up the Master Project:** Create a master Git repository with a clear directory structure for each major service (/lean_engine, /ai_assistant, /frontend).
2. **Containerise the Environment:** Use Docker for containerisation from the start to ensure a consistent development and deployment environment. All services will be defined in a docker-compose.yml file.
3. **Deploy LEAN Engine:** Set up the QuantConnect LEAN engine locally using its Docker image. This will function as your core backtesting and execution service.
4. **Establish the Database:** Define a PostgreSQL service in your docker-compose.yml. Crucially, enable the pgvector extension from the start to handle both traditional relational data and vector embeddings for the AI assistant in a single database.
5. **Connect LEAN to Data:** Configure the LEAN engine to use its built-in data connectors to fetch historical market data from free sources like Yahoo Finance.
6. **Free Data Sources for Paper Trading:**
 - **Stocks & ETFs**
 1. *Interactive Brokers*
 2. *Yahoo Finance*: Free historical and delayed data for global equities and ETFs.
 3. *Alpha Vantage*: Free APIs for real-time and historical data, including fundamentals and technical indicators.
 4. *Finnhub*: Free-tier APIs for real-time stock prices and company fundamentals.
 5. *Twelve Data*
 6. *Polygon.io*
 - **Stock Futures & Index Futures**
 1. *Interactive Brokers*
 2. *Yahoo Finance*: Free historical and delayed data for global equities and ETFs.
 3. *Investing.com*: Free real-time/delayed quotes for global index futures (e.g., S&P 500, Nikkei 225).

4. *CME Group*: Free delayed data for equity index futures (e.g., E-mini S&P 500).
5. *Barchart*: Delayed futures prices for indices like Dow Jones and S&P 500.
- **Stock Options & Index Options**
 1. *Interactive Brokers*
 2. *Yahoo Finance*: Free historical and delayed data for global equities and ETFs.
 3. *Choe Daily Market Statistics*: Free delayed summary data for options trading volumes.
 4. *SpiderRock*: Free delayed options data via APIs (e.g., OPRA feeds).
- **Forex, Forex Futures, & Forex Options**
 1. *Interactive Brokers*
 2. *Yahoo Finance*: Free historical and delayed data for global equities and ETFs.
 3. *Oanda*
 4. *CME Group FX Data*: Delayed data for G10 and emerging market forex futures.
 5. *dxFeed*: Multi-contributor forex data (real-time/delayed).
- **Commodity, Commodity Futures, & Commodity Options**
 1. *Interactive Brokers*
 2. *Yahoo Finance*: Free historical and delayed data for global equities and ETFs.
 3. *TradingCharts*: Free delayed quotes for commodity futures (e.g., crude oil, gold).
 4. *CME Group*: Delayed data for commodities like oil and agricultural futures.
- **Cryptocurrency, Cryptocurrency Futures, & Cryptocurrency Options**
 1. *Interactive Brokers*
 2. *Alpha Vantage*: Free crypto data APIs for Bitcoin, Ethereum, etc.
 3. *Coinbase*
 4. *Binance India*
7. **Data Feed Fallback Mechanism:**
 - Data retrieval follows this order: Yahoo Finance → Alpha Vantage → Finnhub → Investing.com → CME Group → Twelve Data → Polygon → Barchart → SpiderRock → TradingCharts → Oanda. If one source fails, the system automatically switches to the next. This fallback mechanism should be created as per the asset class.
8. **Data Sources for Live Trading:**
 - Subscribe to Interactive Brokers' real-time market data and historical data once paper trading is validated.
9. **Historical Data:**
 - Use free sources for paper trading and
 - Interactive Brokers for live trading.
10. **Options Data Requirements**
 - Historical options chain data (min 5 years depth)
 - Real-time implied volatility surfaces
 - Dividend forecast integration

11. **Test the Core Engine:** Run a simple, pre-built algorithm from the LEAN library to validate that the entire pipeline is working: the engine can fetch data, run a backtest, and generate results.
12. **Trading Instruments:** The system supports a wide range of asset classes to cater to diverse trading needs.
 - Stocks & ETFs (global equities and exchange-traded funds).
 - Stock Futures & Index Futures (e.g., E-mini S&P 500, Nasdaq-100).
 - Stock Options & Index Options (e.g., calls, puts on equities and indices).
 - Forex, Forex Futures, & Forex Options (e.g., EUR/USD, G10 currencies).
 - Commodities, Commodity Futures, & Commodity Options (e.g., crude oil, gold, corn).
 - Cryptocurrency, Cryptocurrency Futures, & Cryptocurrency Options (e.g., Bitcoin, Ethereum).

Features Covered from "Enterprise Algorithmic Trading System...":

- **Provided by Open-Source:**
 - I. Core Trading Infrastructure: Basic setup for multi-broker connectivity (pluggable architecture) and supported asset classes via QuantConnect.
 - II. Data Feed Management: Integration with free data sources like Yahoo Finance via LEAN's connectors.
 - III. Backtesting Engine: The core engine from QuantConnect is now operational for testing strategies against historical data.
 - VI. DevOps: Use of Docker and Git for containerisation and version control. IaC principles established with docker-compose.yml.
- **Custom Development:**
 - I. Order Management: Custom logic for logging and error handling begins here.
 - II. Data Management: Design of custom database tables for storing market data (OHLCV, tick) in PostgreSQL.
 - IV. Database Integration: Setup of PostgreSQL with pgvector for future use by the AI Assistant.
 - VI. Security: Establishment of project structure with security in mind, although no specific security features are implemented yet.

Details for Phase 1:

Phase	Old Phase	Category	Sub-Category	Feature	Implementation Type	Repository / Custom Details	Remarks
1	1	I. Core Trading Infrastructure & Execution	2. Trading Instruments	Comprehensive Asset Class Support (Stocks, ETFs, Futures, Options, Forex, Crypto, etc.)	Foundation Model	Repository: QuantConnect LEAN.	LEAN provides broad asset class support out of the box. The available instruments are determined by the connected broker's offerings.
1	2	II. Data Management & Analytics	1. Data Feed Management	Free Data Sources (Yahoo Finance, Alpha Vantage, etc.)	Foundation Model (Integration)	Repository: QuantConnect LEAN.	LEAN has built-in data connectors for many popular sources, used for initial setup and backtesting.
1	2	II. Data Management & Analytics	1. Data Feed Management	Market data storage tables & historical data archiving	Custom Built	Database: PostgreSQL.	The database schema, storage policies, and archiving logic are custom designs implemented in PostgreSQL.
1	3	III. AI-Powered Strategy Development & Backtesting	1. Strategy Creation Studios & Management	Version Control (Git Integration)	Custom Built (Integration)	Library: GitPython or direct Git CLI.	Git is used from Phase 1, but direct integration into the UI for strategy versioning is a custom feature.
1	3	III. AI-Powered Strategy Development & Backtesting	2. Backtesting & Optimisation Engine	Enterprise Backtester (Core Engine)	Foundation Model	Repository: QuantConnect LEAN.	LEAN provides the core event-driven backtesting engine used from the very first phase.
1	4	IV. The Integrated Agentic AI Assistant	3. Comprehensive Database & Analytics Integration	Relational (PostgreSQL) & Vector (pgvector)	Foundation Model (Integration)	Repository: PostgreSQL with pgvector extension.	This is set up in Phase 1 to serve as the all-in-one database for both relational and vector data.

✅ **End of Phase 1 Goal:** You will have the LEAN engine running in Docker, successfully fetching market data, and saving it to a persistent PostgreSQL database.

Prompt for Phase 2:

Continuing the algorithmic trading system project, your task is to complete Phase 2. The goal is to build an API to control the LEAN engine programmatically.

1. Inside the /lean_engine service directory, create a new **FastAPI** application that will act as a secure bridge to the LEAN engine.
2. Implement a single, critical endpoint: POST /backtest. This endpoint should accept a JSON payload specifying a trading strategy's parameters (e.g., ticker, start/end dates, strategy name).
3. When called, this endpoint must trigger a backtest using the LEAN engine and return the performance results as a JSON object. This JSON should include basic metrics like Sharpe Ratio and Max Drawdown.
4. Add the TA-Lib and pandas-ta libraries to the lean_engine service's Docker environment to make technical indicators available to the strategies.
5. Write a separate Python test script to validate the API by calling the /backtest endpoint and printing the results.
6. Also refer to the following details and decide & plan accordingly.

Phase 2: Building the API Bridge & Initial Backtesting

Objective: To create the critical API layer that will allow all other system components to communicate with the LEAN engine and to enable programmatic backtesting.

Integration Plan & Actions:

1. **Build the FastAPI Wrapper:** Inside the /lean_engine service, create a new FastAPI application. This API will serve as the central, secure "bridge" to the LEAN engine, abstracting its complexity.
2. **Create the Backtesting Endpoint:** Implement the first major endpoint: POST /backtest. This endpoint will accept a strategy configuration (e.g., as a JSON object), trigger the LEAN engine to run the simulation, and return the performance results.
3. **Integrate Indicator Libraries:** Add essential Python libraries like TA-Lib and pandas-ta to the backend environment so that strategies executed by LEAN can easily compute hundreds of technical indicators.
4. **Develop a Test Script:** Write a simple Python script to call the new /backtest endpoint. This validates that the API bridge is functioning correctly and decouples the engine from other services.

Features Covered from "Enterprise Algorithmic Trading System...":

- **Provided by Open-Source:**
 - III. Technical Indicators: Integration of TA-Lib and pandas-ta provides a comprehensive library of indicators.
- **Custom Development:**
 - I. Order Management: The API bridge is the first step towards a full Order Management System. The backtest endpoint simulates execution.
 - III. Backtesting Engine: While LEAN is the engine, the custom API allows for programmatic control, a key requirement.
 - IV. Post-Trade Analytics: The JSON results from the backtest endpoint will form the basis for future TCA and performance analysis. You can include basic metrics like Sharpe Ratio and Max Drawdown here.
 - X. Integration & Orchestration Layer: The FastAPI application is the first version of this crucial layer.

Details for Phase 2:

Phase	Old Phase	Category	Sub-Category	Feature	Implementation Type	Repository / Custom Details	Remarks
2	1	I. Core Trading Infrastructure & Execution	3. Order Management System	Order execution logging	Custom Built	Library: Python's logging module.	This is a custom application logic feature to record all API requests and responses for auditing and analysis.
2	3	III. AI-Powered Strategy Development & Backtesting	2. Backtesting & Optimisation Engine	Realistic Modeling (Slippage, Commissions, etc.)	Foundation Model	Repository: QuantConnect LEAN.	LEAN's backtester includes models for slippage and fees, which are configured during backtest setup.

✅ **End of Phase 2 Goal:** You will have a custom API that can programmatically trigger a backtest in the LEAN engine and retrieve the results.

Prompt for Phase 3:

Your task is to build the AI Assistant for the trading system as defined in Phase 3. The goal is to connect a conversational AI to the trading engine's API.

1. In the /ai_assistant directory, set up a new Python service.
2. Use **LangChain** as the core framework. Integrate the **TradingAgents** library to provide specialised financial agent roles. Configure it to use a local LLM via **Ollama**.
3. Build an Agentic RAG (Retrieval-Augmented Generation) pipeline:
 - o Use the **Unstructured.io** library to process uploaded documents (PDFs, DOCX).
 - o Store the document embeddings in the PostgreSQL/pgvector/Qdrant database.
4. Using LangChain, create two initial AI 'tools':
 - o run_backtest_tool: This tool makes a POST request to the /backtest API endpoint created in Phase 2.
 - o query_documents_tool: This tool performs a similarity search against the vector database.
5. For the user interface, configure a pre-built **Open-WebUI** Docker container to connect to your LangChain backend. The final deliverable is an AI chatbot that can answer questions from documents and run a backtest via a text command.
6. Also refer to the following details and decide & plan accordingly.

Phase 3: Bringing the AI Assistant to Life

Objective: To build the foundational layer of the Agentic AI Assistant and connect it to the trading engine through the newly created API bridge.

Integration Plan & Actions:

1. **Set Up the AI Assistant Service:** In the /ai_assistant directory, set up the project structure for the AI components.
2. **Integrate LangChain & TradingAgents:** Use LangChain as the core framework to manage prompts, interact with LLMs (via Ollama for local models), and build the Agentic RAG pipeline. Integrate the TradingAgents project to provide the specialised financial agent roles.
3. **Build the Agentic RAG Pipeline:**
 - o **Document Processing:** Use the Unstructured.io library to process uploaded documents (PDFs, DOCX, etc.) into clean text.
 - o **Supported Formats:**
 - Text: PDF, DOCX, RTF, TXT, MD, HTML, HTM, XML, JSON, YAML, YML, TOML, INI, and LOG files.
 - Presentations: PPTX, PPT, ODP, KEY, Google Slides (via API).
 - Spreadsheets: XLSX, XLS, CSV, TSV, ODS, NUMBERS, Google Sheets (via API).
 - Specialised: EPUB, MOBI, LaTeX, ReStructuredText, AsciiDoc, Jupyter Notebooks, Confluence exports.
 - Archives: ZIP, RAR, 7Z, TAR (with recursive extraction and malware scanning).
 - Images (OCR): PNG, JPEG, JPG, TIFF, BMP, WEBP, SVG, with advanced layout detection.
 - Audio/Video: MP4, AVI, MOV, MP3, WAV, FLAC (transcription via Whisper), with speaker identification.
 - Multi-Modal: PDF with OCR for text and images, with table extraction and chart recognition
 - New Formats: CAD files (metadata), Email files (EML, MSG), and Database exports (SQL dumps).
 - o **Vector Storage:** Use the Qdrant Python client to store the resulting text embeddings in your vector database.
4. **Create Initial AI "Tools":** This is the critical integration step. Using LangChain, create custom "tools" that the AI agents can use. The first two essential tools will be:
 - o run_backtest_tool: This tool makes a POST request to the /backtest endpoint on your FastAPI bridge.
 - o query_documents_tool: This tool performs a similarity search against the Qdrant database to answer questions from ingested documents.
5. **Deploy a Chat UI:** For rapid prototyping, deploy Open-WebUI as a pre-built, user-facing chat interface that can connect to your LangChain-based AI backend.

Features Covered from "Enterprise Algorithmic Trading System...":

- **Provided by Open-Source:**
 - IV. Core Intelligence: LangChain provides the Agentic RAG framework. Ollama enables use of local LLMs. TradingAgents provides the collaborative agent network foundation.
 - IV. Document Processing: Unstructured.io for universal ingestion of various file formats.
 - IV. Database Integration: Qdrant integration for vector search.
 - V. Conversational Chatbot Interface: Use of Open-WebUI for the initial UI.
- **Custom Development:**
 - III. Generative AI Assistant: The core logic for converting plain text prompts ("Run a backtest...") into API calls is developed here.
 - IV. AI Tools: The run_backtest_tool and query_documents_tool are the first custom agent tools, forming the bridge between the AI and the trading system.
 - IV. Hybrid Search: The query_documents_tool is the first part of the advanced search capability.

Details for Phase 3:

Phase	Old Phase	Category	Sub-Category	Feature	Implementation Type	Repository / Custom Details	Remarks
3	3	III. AI-Powered Strategy Development & Backtesting	1. Strategy Creation Studios & Management	Generative AI Assistant (for strategy code)	Hybrid	Foundation: LangChain, TradingAgents. Custom Logic: A specialised prompt/tool for code generation.	This is a key feature of the AI assistant, which will be trained to generate Python code compatible with the LEAN engine.
3	4	IV. The Integrated Agentic AI Assistant	1. Core Intelligence: Agentic Framework	Agentic RAFT System & RAG	Foundation Model (Integration)	Repository: LangChain.	LangChain is the core framework used to build the entire RAG pipeline and agentic system.
3	4	IV. The Integrated Agentic AI Assistant	1. Core Intelligence: Agentic Framework	Local LLMs	Foundation Model (Integration)	Repository: Ollama.	Ollama is used to easily serve open-source LLMs locally, integrating seamlessly with LangChain.
3	4	IV. The Integrated Agentic AI Assistant	2. Comprehensive Document & Content Processing	Universal Ingestion (All file formats)	Foundation Model (Integration)	Repository: Unstructured.io.	Unstructured.io is purpose-built to parse a vast array of complex file formats into clean text for the RAG pipeline.
3	4	IV. The Integrated	2. Comprehensive Document &	Intelligent Processing Pipeline	Hybrid	Foundation: Unstructured.io, LangChain.	The libraries provide the core functions, but the pipeline that connects them (extraction, chunking, embedding) is built using LangChain.

		Agentic AI Assistant	Content Processing			Custom Logic: For orchestration.	
3	4	IV. The Integrated Agentic AI Assistant	3. Comprehensive Database & Analytics Integration	Vector (Qdrant)	Foundation Model (Integration)	Repository: Qdrant.	Qdrant is integrated in Phase 3 as the high-performance vector database for the RAG system.
3	4	IV. The Integrated Agentic AI Assistant	5. Collaborative Agent Network (A2A Protocol)	A Network of Specialists	Hybrid	Foundation: TradingAgents, LangChain.	- Custom Logic: For specific agent roles and collaboration. - TradingAgents provides the finance-specific roles. - LangChain provides the framework. The specific orchestration and advanced roles are custom.
3	5	V. User Experience & Multi-Platform Accessibility	2. Conversational Chatbot Interface	UI/UX (Modern Chat Interface)	Hybrid	Foundation: Open-WebUI for prototype.	- Custom Built: Final version in Next.js. - Open-WebUI is used for rapid prototyping in Phase 3. - The final, deeply integrated chat UI will be custom-- built within the main Next.js application in Phase 4/5.

✅ **End of Phase 3 Goal:** A user can interact with a web-based chatbot, ask questions about uploaded documents (Agentic RAG), and issue a command like, "Run a backtest on the simple moving average strategy for AAPL," which the AI agent executes by calling the trading engine's API.

Prompt for Phase 4:

Your task is to build the front-end and enable paper trading and live trading as described in Phase 4.

1. In the /frontend directory, set up a new web application using **Next.js**, React, and TypeScript.
2. Integrate the **TradingView Lightweight Charts** library into the Next.js app to display market data.
3. Expand the FastAPI bridge with new endpoints for live trading: GET /portfolio, POST /order, and GET /orders. These endpoints will interact with the LEAN engine.
4. Configure the LEAN engine's built-in **Interactive Brokers** integration to connect to a paper trading and a live trading account.
5. Create new LangChain tools for the AI assistant that use these new endpoints, allowing a user to ask, 'What is my current portfolio?' or command, 'Place a limit order to buy 10 shares of GOOG at \$150.' The goal is a unified web app where a user can view charts and command the AI to execute paper trades as well as live trades.
6. Also refer to the following details and decide & plan accordingly.

Phase 4: Full User Experience, Paper Trading & Live Trading

Objective: To build the polished, custom front-end and connect the entire integrated system to a paper trading and live trading account.

Integration Plan & Actions:

1. **Build the Custom Next.js Frontend:** In the /frontend directory, begin building your custom user interface based on Next.js, React, and TypeScript, as specified in your features document.
2. **Integrate Advanced Charting:** Use the TradingView Lightweight Charts library to display market data and backtest results. This data will be fetched from new endpoints on your FastAPI bridge.
3. **Enable Paper Trading and Live Trading in LEAN:** Configure LEAN's pre-built Interactive Brokers integration to connect to a paper and a live trading account.
 - o **Paper Trading:** A key objective of Phase 4 is to connect the entire integrated system to a paper trading account. The integration plan for this phase explicitly includes configuring the LEAN engine's pre-built Interactive Brokers integration to connect to a paper trading account.
 - o **Live Trading:** Phase 4 also introduces live trading capabilities. The plan includes enabling live trading in the LEAN engine, expanding the API to manage live trading operations, and enabling live trading data feeds. The end goal of this phase is for a user to execute and monitor trades in a live trading account through the custom web application.
4. **Expand the API Bridge:** Add new endpoints to your FastAPI wrapper to manage live trading operations. Essential endpoints include GET /portfolio, POST /order, GET /orders, and DELETE /order/{order_id}.
5. **Create Advanced AI Tools:** Empower your AI agents by creating new LangChain tools that consume these new live trading endpoints. This allows users to issue commands like, "What is my current portfolio?" or "Place a limit order to buy 10 shares of GOOG at \$150".

Features Covered from "Enterprise Algorithmic Trading System...":

- **Provided by Open-Source:**
 - o I. Broker Connectivity: QuantConnect's native Interactive Brokers integration is enabled.
 - o V. Institutional Charting: TradingView Lightweight Charts library integrated.
- **Custom Development:**
 - o I. Order Management: Full implementation of the OMS through the API for all basic and advanced order types supported by the broker.
 - o V. Unified User Interface: Development of the custom Next.js application, including personalised dashboards, widgets, and visualisations.
 - o V. Mobile & Web Access: The Next.js app will be responsive, and PWA capabilities can be enabled.
 - o IX. Portfolio Management: The GET /portfolio endpoint and corresponding UI components provide real-time tracking of P&L and positions.
 - o IX. Alerts and Notifications: The backend can now be architected to support multi-channel alerts for trade executions and order status changes.

Details for Phase 4:

Phase	Old Phase	Category	Sub-Category	Feature	Implementation Type	Repository / Custom Details	Remarks
4	1	I. Core Trading Infrastructure & Execution	1. Multi-Broker & Market Connectivity	Direct integration with Interactive Brokers	Foundation Model (Integration)	Repository: QuantConnect LEAN.	LEAN has a pre-built, robust integration for Interactive Brokers. Phase 4 focuses on enabling this for paper trading and live trading.
4	1	I. Core Trading Infrastructure & Execution	1. Multi-Broker & Market Connectivity	Real-time connection health monitoring	Custom Built	Custom Logic: Built around the broker SDKs.	This involves custom logic to handle API heartbeats, exceptions, and connection state management.
4	1	I. Core Trading Infrastructure & Execution	1. Multi-Broker & Market Connectivity	Seamless switching between paper trading and live trading accounts	Custom Built	Custom Logic: Implemented in the API bridge and frontend.	This is a custom feature built into the platform's core logic to manage different broker configurations.
4	1	I. Core Trading Infrastructure & Execution	3. Order Management System	Basic Order Types (Market, Limit, Stop, etc.)	Foundation Model	Repository: QuantConnect LEAN.	These order types are natively supported by LEAN and the underlying broker APIs.
4	1	I. Core Trading Infrastructure & Execution	3. Order Management System	Advanced Order Types (GTC, IOC, FOK)	Foundation Model	Repository: QuantConnect LEAN.	These are generally supported by broker APIs and can be accessed through LEAN.
4	1	I. Core Trading Infrastructure & Execution	3. Order Management System	Error handling for failed orders	Custom Built	Custom Logic: Within the API Bridge/Execution Service.	Automated responses like retries, notifications, or fallback routing are custom business logic.
4	2	II. Data Management & Analytics	1. Data Feed Management	Live Trading Data (via Interactive Brokers)	Foundation Model (Integration)	Repository: QuantConnect LEAN.	This is enabled when the live trading connection to Interactive Brokers is configured in Phase 4.
4	5	V. User Experience & Multi-Platform Accessibility	1. Unified User Interface (UI)	Core Framework (Next.js, React, TypeScript)	Foundation Model (Framework)	Framework: Next.js.	Next.js is the chosen front-end framework for building the entire custom user interface.
4	5	V. User Experience & Multi-Platform Accessibility	1. Unified User Interface (UI)	Personalised Dashboards & Widgets	Custom Built	- Frontend: Next.js. - Libraries: MUI or shadcn/ui.	The entire dashboard system, including drag-and-drop widgets, is a custom front-end development effort.

4	5	V. User Experience & Multi-Platform Accessibility	1. Unified User Interface (UI)	Institutional Charting (TradingView)	Foundation Model (Integration)	Library: TradingView Lightweight Charts.	This powerful charting library is integrated into the custom Next.js frontend to provide a familiar experience.
4	5	V. User Experience & Multi-Platform Accessibility	3. Mobile, Desktop & Web Access	Web Application (Responsive)	Custom Built	Framework: Next.js.	The web application is the primary interface, built to be responsive from the start.
4	7	VII. Options Trading	6. Operational Monitoring & DevOps	Basic & Complex Strategy Support	Foundation Model	Repository: QuantConnect LEAN.	LEAN supports building and backtesting multi-leg options strategies like Iron Condors and Butterflies.

✅ **End of Phase 4 Goal:** A fully functional, integrated system where a user can log into your custom web application, view charts, chat with an AI assistant, and command that assistant to execute and monitor trades in a paper trading and live trading account.

Prompt for Phase 5:

Your task is to implement the advanced, enterprise-grade features from Phase 5. This is a large-scale integration effort.

1. **Options Analytics:** Create a new microservice or expand the FastAPI bridge to use the **QuantLib** library. Expose endpoints for calculating option Greeks and pricing. Create an AI tool so the assistant can answer questions about options.
2. **No-Code Builder:** Build the custom no-code strategy builder.
 - **Frontend:** In the Next.js app, create a drag-and-drop UI that outputs a strategy as a JSON object.
 - **Backend:** Write a Python interpreter that parses this JSON and constructs a strategy that the LEAN engine can execute.
3. **Enterprise Guardrails:** In the FastAPI bridge, implement pre-trade risk checks (e.g., concentration risk) that intercept all POST /order requests before they reach the LEAN engine.
4. **Observability:** Add **Prometheus** and **Grafana** to the docker-compose.yml file. Instrument the FastAPI bridge to export key metrics (e.g., API latency, order count) to Prometheus for visualisation in a Grafana dashboard.
5. Also refer to the following details and decide & plan accordingly.

Phase 5: Advanced Features & Enterprise Readiness

Objective: To layer in the final, sophisticated features that differentiate the platform, focusing on advanced analytics, risk management, security, and operational monitoring.

Integration Plan & Actions:

1. **Integrate QuantLib for Advanced Options Analytics:** Create a new microservice (or expand the FastAPI bridge) that uses the QuantLib library. This will expose new API endpoints for advanced options calculations (Greeks, volatility surfaces, pricing for exotics), which can then be surfaced to both the UI and the AI agents as a new tool.
2. **Build the No-Code Strategy Builder:**
 - **Frontend:** In the Next.js app, build the visual drag-and-drop interface that allows users to create strategies by connecting logic blocks. This UI will generate a JSON configuration file.
 - **Backend:** Write a custom interpreter module that parses this JSON and programmatically constructs a strategy that the LEAN engine can execute. This is a significant piece of custom development.
3. **Implement Enterprise Guardrails:**
 - **Risk Management:** Build the real-time risk hub logic into the FastAPI wrapper. These pre-trade checks (e.g., for account-level circuit breakers, concentration risk) will be called before any order is sent to the LEAN engine.
 - **Security & Compliance:** Implement the Zero-Trust architecture principles, RBAC, and immutable audit trails. The Compliance Agent can be built into the AI network.
4. **Deploy Full Observability Stack:** Add Prometheus (for metrics) and Grafana (for dashboards) to your docker-compose.yml file. Instrument all backend services to export metrics, creating a comprehensive production monitoring dashboard.
5. **Implement Remaining Advanced Features:** This is the stage for building out the full suite of differentiating features:
 - **Data & Analytics:** Market microstructure analysis, TCA, AI-powered opportunity detection.
 - **AI & Backtesting:** Adversarial testing, AI overfitting detector, hyperparameter tuning with Optuna.
 - **AI Assistant:** The full collaborative agent network (Orchestrator, Compliance, etc.), continuous learning pipeline, and advanced hybrid search.
 - **Workflow Orchestration:** Integrate LangGraph to manage complex, asynchronous workflows between the AI Agents, Multi Agents, Agentic AI, data, and trading services.

Features Covered from "Enterprise Algorithmic Trading System...":

- **Provided by Open-Source:**

- VII. Options Trading: QuantLib integration for all advanced pricing, volatility, and risk analytics.
- VI. Operational Monitoring: Prometheus and Grafana for the full observability stack.

- **Custom Development:**

All remaining features from the document are implemented here. This includes, but is not limited to:

- II. Advanced Data: News sentiment analysis, data resiliency, market microstructure analysis.
- III. Strategy Creation: The complete No-Code Builder, AI-assisted optimisation, and advanced validation techniques.
- IV. AI Assistant: The full implementation of the specialist agent network, custom fine-tuning, and intelligent model routing.
- VI. Security & Compliance: The complete suite of enterprise security, compliance, risk management, and RBAC features.
- VII. Options Trading: The UI/UX for the strategy builders, simulators, and dashboards that consume the QuantLib analytics.
- IX. Additional Features: Portfolio X-Ray analytics, AI Trading Coach, Cross-Asset Arbitrage Engine, etc.

Details for Phase 5:

Phase	Old Phase	Category	Sub-Category	Feature	Implementation Type	Repository / Custom Details	Remarks
5	1	I. Core Trading Infrastructure & Execution	1. Multi-Broker & Market Connectivity	Expanded connectivity to other brokers (Alpaca, TD Ameritrade)	Foundation Model (Integration)	Repository: QuantConnect LEAN.	LEAN supports numerous brokers. Integration of additional brokers would occur as needed in a later phase.
5	1	I. Core Trading Infrastructure & Execution	1. Multi-Broker & Market Connectivity	Support for the institutional FIX protocol	Custom Built (Integration)	Library: quickfix engine.	This requires custom development to integrate the quickfix library into the trading engine service.
5	1	I. Core Trading Infrastructure & Execution	3. Order Management System	Algorithmic Orders (VWAP, TWAP, Iceberg, OCO)	Hybrid	Foundation: QuantConnect LEAN for the execution framework.	- Custom Logic: The core VWAP/TWAP logic for breaking down parent orders will be custom Python code. - While some brokers offer these natively, building the logic yourself provides greater control and portability.
5	2	II. Data Management & Analytics	1. Data Feed Management	Data Resiliency (Fallback Mechanism)	Custom Built	Custom Logic: A data abstraction layer.	This is a custom-built layer responsible for querying different API sources in a prioritised sequence.
5	2	II. Data Management & Analytics	1. Data Feed Management	Real-time Data Quality Scoring & Smart Data Fusion	Custom Built	Custom Logic: Custom algorithms.	These are advanced, proprietary features requiring custom algorithms to check data quality and create composite feeds.

5	2	II. Data Management & Analytics	1. Data Feed Management	Data Latency Monitoring	Custom Built	Custom Logic: Monitoring and alerting system.	This will be custom logic to track data delivery times and trigger alerts, likely feeding into Prometheus.
5	2	II. Data Management & Analytics	2. Alternative & Advanced Data Integration	Real-time News Feeds & Multi-Source Aggregation	Custom Built (Integration)	APIs: NewsAPI, Finnhub, etc..	This involves integrating third-party news APIs and building custom logic for aggregation and scoring.
5	2	II. Data Management & Analytics	2. Alternative & Advanced Data Integration	Advanced Sentiment Analysis	Hybrid	Foundation: Hugging Face Transformers library.	- Custom Logic: For integration and event-driven alerting. - Using a pre-trained model like FinBERT provides state-of-the-art NLP, while the surrounding application is custom.
5	2	II. Data Management & Analytics	2. Alternative & Advanced Data Integration	News Impact Modeling	Custom Built	Libraries: scikit-learn, PyTorch.	This is a custom quantitative modeling feature to correlate news events with market effects.
5	2	II. Data Management & Analytics	2. Alternative & Advanced Data Integration	Fundamental, Economic, ESG, and Exotic Data Integration	Custom Built (Integration)	APIs: FRED, specific data vendors.	This involves writing custom connectors for various third-party data vendor APIs.
5	2	II. Data Management & Analytics	3. Market Analytics & Microstructure	Advanced Market Screener/Scanner	Custom Built	Custom Logic: Backend filtering logic.	The core filtering and analysis logic is custom, built on top of the integrated data feeds.
5	2	II. Data Management & Analytics	3. Market Analytics & Microstructure	Natural Language Filters	Hybrid	Foundation: LangChain and an LLM. Custom Logic: A tool for the AI agent.	The AI Assistant will parse the natural language and translate it into a structured query for the custom-built scanner.
5	2	II. Data Management & Analytics	3. Market Analytics & Microstructure	AI-Powered Opportunity Detection	Custom Built	Libraries: scikit-learn, PyTorch.	This is a custom machine learning model trained to identify unusual market patterns.
5	2	II. Data Management & Analytics	3. Market Analytics & Microstructure	Market Microstructure Analysis (Order Book, etc.)	Custom Built	Custom Logic: Real-time analysis algorithms.	This requires custom logic to process high-resolution tick data and analyse order book dynamics.
5	2	II. Data Management & Analytics	4. Post-Trade Analytics	Transaction Cost Analysis (TCA), Fill Rate, Broker Performance	Custom Built	Libraries: Pandas, NumPy.	This is primarily custom analysis logic that runs on the trade execution logs stored in your database.

5	3	III. AI-Powered Strategy Development & Backtesting	1. Strategy Creation Studios & Management	No-Code Strategy Builder	Custom Built	Frontend: Next.js.	- Backend: Custom JSON interpreter. - The visual UI and the backend logic to translate it into an executable strategy are entirely custom.
5	3	III. AI-Powered Strategy Development & Backtesting	1. Strategy Creation Studios & Management	Python/API Studio (Cloud Jupyter)	Custom Built (Integration)	Integration: JupyterHub or similar services.	Integrating a managed notebook environment into the platform requires custom work. The AI Copilot would be part of the AI Assistant.
5	3	III. AI-Powered Strategy Development & Backtesting	2. Backtesting & Optimisation Engine	Advanced Analytics (Monte Carlo, Walk-Forward)	Custom Built	Custom Logic: Python scripts using the backtester.	These are advanced validation techniques that require custom analytical code to be written around the core backtesting engine.
5	3	III. AI-Powered Strategy Development & Backtesting	2. Backtesting & Optimisation Engine	AI-Assisted Optimisation (Hyperparameter Tuning)	Custom Built (Integration)	Library: Optuna.	Optuna is a best-in-class library that will be integrated to work with the LEAN backtester for optimisation tasks.
5	3	III. AI-Powered Strategy Development & Backtesting	2. Backtesting & Optimisation Engine	AI Overfitting Detector	Custom Built	Libraries: scikit-learn.	This is a custom machine learning model that analyses backtest results to detect curve-fitting.
5	4	IV. The Integrated Agentic AI Assistant	1. Core Intelligence: Agentic Framework	Pre-Tuned Domains & Custom Fine-Tuning	Custom Built	Custom Logic: MLOps pipeline.	This involves building a custom pipeline for managing datasets and initiating fine-tuning jobs on LLMs.
5	4	IV. The Integrated Agentic AI Assistant	1. Core Intelligence: Agentic Framework	Intelligent Model Routing	Custom Built	Foundation: LangChain. Custom Logic: A router chain.	This is a custom "router" built within LangChain to direct queries to the most appropriate model or agent.
5	4	IV. The Integrated Agentic AI Assistant	1. Core Intelligence: Agentic Framework	Continuous Learning Pipeline	Custom Built	Custom Logic: Feedback loop mechanism.	This is an advanced MLOps feature requiring a custom pipeline to collect user feedback and trigger model retraining.

5	4	IV. The Integrated Agentic AI Assistant	3. Comprehensive Database & Analytics Integration	Other Databases (NoSQL, Time-Series, etc.)	Custom Built (Integration)	Custom Connectors: LangChain connectors.	Integration with other database types would be done on an as-needed basis by building custom LangChain tools/connectors.
5	4	IV. The Integrated Agentic AI Assistant	4. Advanced Hybrid Search & Retrieval	Multi-Modal Search & Advanced Ranking	Custom Built	Custom Logic: Within the LangChain retrieval chain.	This involves custom logic for combining different search methods (semantic, keyword) and implementing advanced ranking algorithms.
5	4	IV. The Integrated Agentic AI Assistant	4. Advanced Hybrid Search & Retrieval	Personalised & Federated Search	Custom Built	Custom Logic: Custom agent/tool.	This requires custom development to manage user profiles and search across multiple data sources simultaneously.
5	4	IV. The Integrated Agentic AI Assistant	4. Advanced Hybrid Search & Retrieval	Web Search Integration & Fact-Checking	Custom Built (Integration)	APIs: Google, Bing, etc.	A custom LangChain tool would be created to call external search APIs and process the results.
5	5	V. User Experience & Multi-Platform Accessibility	2. Conversational Chatbot Interface	Long-Term Memory & Conversational Intelligence	Custom Built	Foundation: LangChain. Custom Logic: Conversation history management.	This is implemented using LangChain's memory modules, but the specific logic for long-term, cross-session context is a custom implementation.
5	5	V. User Experience & Multi-Platform Accessibility	3. Mobile, Desktop & Web Access	Mobile Apps (iOS & Android)	Custom Built	Framework: React Native.	Building native mobile apps is a separate development effort, but React Native allows for significant code sharing with the web app.
5	5	V. User Experience & Multi-Platform Accessibility	3. Mobile, Desktop & Web Access	Progressive Web App (PWA)	Custom Built	Framework: Next.js.	This feature is enabled through the configuration and service worker capabilities of the Next.js framework.
5	6	VI. Enterprise-Grade Security, Compliance & Operations	1. Advanced Security & Compliance	Zero-Trust Architecture	Custom Built	Custom Logic: Architectural principle.	This is not a library but an architectural approach that is custom-built into the application's core request/authentication flow.
5	6	VI. Enterprise-Grade Security, Compliance & Operations	1. Advanced Security & Compliance	Container Security (Scanning & Runtime)	Foundation Model (Integration)	Repositories: OpenVAS (scanning), Wazuh (runtime).	These open-source security tools are integrated into the CI/CD pipeline and deployment environment.

5	6	VI. Enterprise-Grade Security, Compliance & Operations	2. Regulatory Compliance & Auditing	Automated Compliance & Audit Trails	Custom Built	Custom Logic: Logging and reporting modules.	This involves custom logic for generating immutable logs and creating reports to meet standards like GDPR.
5	6	VI. Enterprise-Grade Security, Compliance & Operations	2. Regulatory Compliance & Auditing	Compliance Agent	Hybrid	Foundation: TradingAgents, LangChain. Custom Logic: A specialised agent.	A new, custom-defined agent is added to the AI network with rules to ensure compliance.
5	6	VI. Enterprise-Grade Security, Compliance & Operations	4. Risk Management	Real-Time Risk Hub (Pre-trade checks, circuit breakers)	Custom Built	Custom Logic: Implemented in the FastAPI API Bridge.	These critical pre-trade checks are custom logic that intercepts orders before they are sent to the LEAN engine.
5	6	VI. Enterprise-Grade Security, Compliance & Operations	4. Risk Management	Advanced Risk Analytics (VaR, Correlation, etc.)	Custom Built	Libraries: Pandas, NumPy, SciPy.	These are custom financial models and calculations built to run on portfolio and market data.
5	6	VI. Enterprise-Grade Security, Compliance & Operations	5. User Management & Entitlements	Role-Based Access Control (RBAC)	Custom Built	Custom Logic: User and permissions system.	This is a standard but custom-built feature for managing user roles and permissions within the application.
5	6	VI. Enterprise-Grade Security, Compliance & Operations	6. Operational Monitoring & DevOps	Observability Stack (Prometheus, Grafana, ELK)	Foundation Model (Integration)	Repositories: Prometheus, Grafana, ELK stack.	These are the industry-standard open-source tools for monitoring, integrated into the Docker environment in Phase 5.
5	6	VI. Enterprise-Grade Security, Compliance & Operations	6. Operational Monitoring & DevOps	Infrastructure as Code (IaC)	Foundation Model (Tooling)	Tools: Terraform, Ansible, Kubernetes manifests.	The infrastructure is managed using these standard IaC tools, ensuring repeatability and consistency.
5	6	VI. Enterprise-Grade Security, Compliance & Operations	6. Operational Monitoring & DevOps	CI/CD Pipeline	Custom Built	Tools: GitHub Actions, Jenkins, etc.	A custom pipeline is built using standard CI/CD tools to automate building, testing, and deployment.
5	7	VII. Options Trading	6. Operational Monitoring & DevOps	Analysis Tools (Greeks, Payoff Diagrams)	Hybrid	Foundation: QuantLib for calculations.	- Custom Built: UI in Next.js. - QuantLib provides the heavy lifting for all Greeks calculations. The UI to display these calculations and payoff diagrams is custom.

5	7	VII. Options Trading	6. Operational Monitoring & DevOps	Exotics Pricing & Advanced Models (Heston, etc.)	Foundation Model (Integration)	Repository: QuantLib.	QuantLib is the premier open-source library for pricing exotic options and using advanced models.
5	7	VII. Options Trading	6. Operational Monitoring & DevOps	Volatility Suite (Surface Constructor, Skew Detector)	Hybrid	Foundation: QuantLib.	- Custom Built: Visualisation UI in Next.js. - QuantLib provides the tools for volatility modeling. The 3D visualisations and heatmaps are custom front-end components.
5	7	VII. Options Trading	6. Operational Monitoring & DevOps	Risk Management (Greek Exposure, Margin Forecaster)	Custom Built	Foundation: QuantLib for data. Custom Logic: Dashboard and forecasting models.	The dashboards and forecasting tools are custom features that consume the underlying Greek data provided by QuantLib.
5	7	VII. Options Trading	6. Operational Monitoring & DevOps	Execution & Liquidity Intelligence	Custom Built	Custom Logic: Custom algorithms and AI tools.	Features like the Legging Optimiser and Dark Pool Scanner are advanced, proprietary algorithms.
5	7	VII. Options Trading	6. Operational Monitoring & DevOps	Automation & Advanced Tools (Auto-Roll, IV Trigger)	Custom Built	Custom Logic: Custom backend services.	These are automated trading bots that run as custom services, executing trades via the API bridge based on specific rules.

✅ **End of Phase 5 Goal:** A feature-complete, secure, and monitorable enterprise-grade algorithmic trading platform, ready for its first users.